

Cloud Security & Architecture – Unit 2 Project Report

Problem-Based Learning (PBL) Assessment

Student Name: Saif Pathan

Course: Cloud Security & Architecture

Deliverable: Unit 2 - IAM & Least Privilege Enforcement

Repository: <https://github.com/saifpathan9969/CSA-UNIT-2->

Related Project (Unit 1): <https://github.com/saifpathan9969/CSA>

Date: September 2026

Executive Summary

This report documents the architectural design, security policy engineering, and empirical verification for **Unit 2** of the Cloud Security Problem-Based Learning (PBL) curriculum: **IAM & Least Privilege Enforcement**.

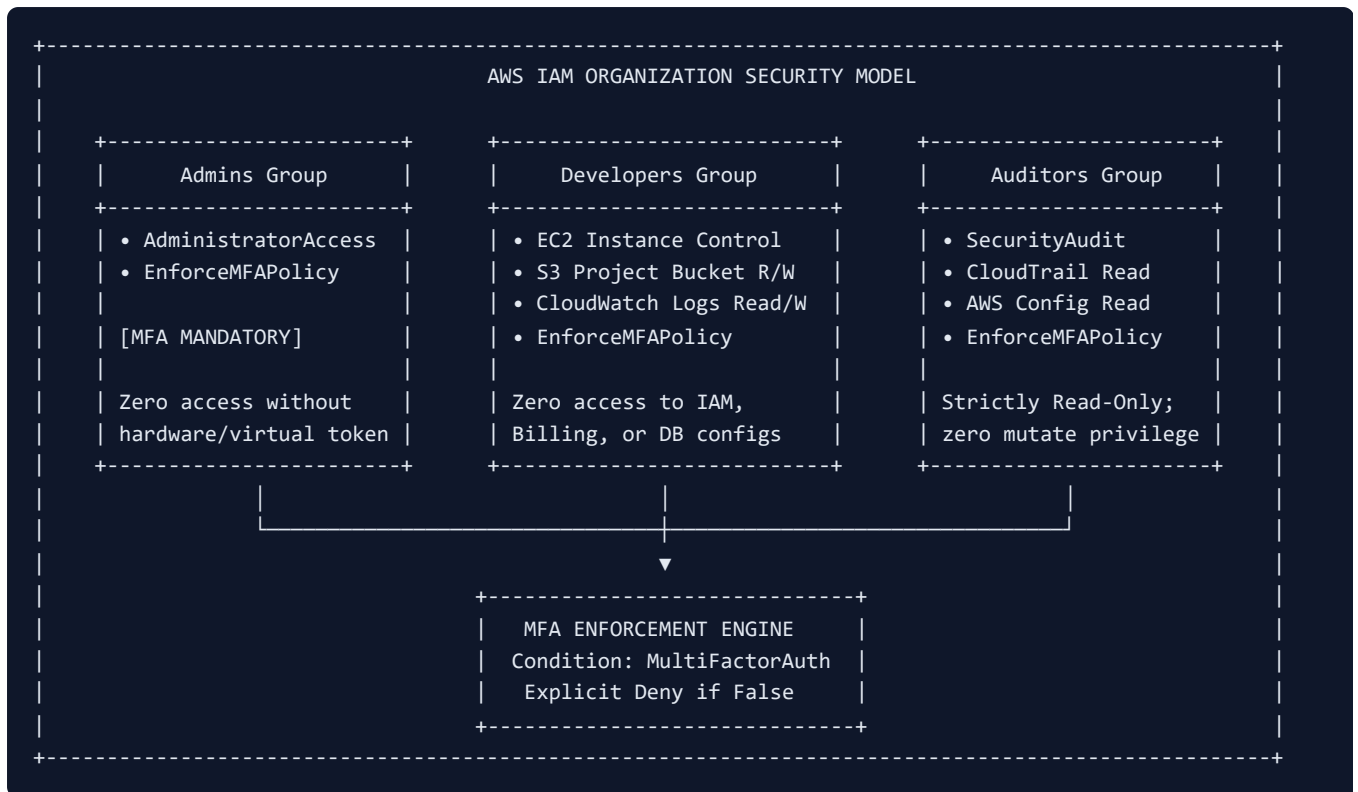
Identity and Access Management (IAM) represents the primary perimeter in modern cloud architectures. This project delivers an enterprise-grade IAM security architecture implemented through **Terraform Infrastructure as Code (IaC)**, implementing:

1. **Three Distinct IAM User Groups** (**Admins** , **Developers** , **Auditors**) governed by custom policies engineered under the **Principle of Least Privilege (PoLP)**.
2. **Cryptographic Multi-Factor Authentication (MFA) Enforcement** with conditional denial (`aws:MultiFactorAuthPresent: true`) across all administrative and operational API endpoints.
3. **Credential-Free EC2-to-S3 Role Delegation** via an IAM Role and Instance Profile, utilizing automated STS temporary token delivery and IMDSv2 to eliminate long-lived access keys.
4. **Hardened S3 Cloud Storage** enforcing AES-256 KMS encryption at rest and mandatory TLS 1.2+ encrypted transit.

Part 1: IAM Architecture & Group Hierarchy

1.1 Architectural Model & Access Control Philosophy

The IAM structure follows a strict **Role-Based Access Control (RBAC)** paradigm combined with **Attribute-Based Access Control (ABAC)** conditions:



1.2 Group Permission Matrix & Least Privilege Mapping

Group Name	Intended Role & Persona	Attached Policies	Allowed Actions	Denied Actions (Implicit / Explicit)
Admins	Cloud Infrastructure Administrators & Security Leads	- AdministratorAccess - EnforceMFAPolicy	All AWS API calls across all services	Explicit Deny on all actions if MFA is not authenticated (<code>aws:MultiFactorAuthPresent == false</code>).
Developers	Backend Engineers & Application Developers	- DeveloperLeastPrivilegePolicy - EnforceMFAPolicy	- EC2: Start, Stop, Reboot, Describe - S3: <code>GetObject</code> , <code>PutObject</code> , <code>ListBucket</code> (Scoped to project bucket ARN only) - CloudWatch: <code>PutLogEvents</code>, <code>GetLogEvents</code>	- IAM: No user/policy creation - EC2: Cannot terminate production instances - S3: Cannot delete bucket or access other buckets - Billing: Zero access
Auditors	Compliance Officers & Security Auditors	- SecurityAudit (AWS Managed) - AuditorCompliancePolicy - EnforceMFAPolicy	Read-only inspection across IAM, CloudTrail, AWS Config, GuardDuty, KMS key metadata	Explicit Deny on any write, update, delete, or create API calls across any AWS service.

1.3 IAM Architecture Diagram

Below is the technical architectural diagram detailing the group permissions, MFA enforcement logic, and credential delegation flows:

IAM Architecture & Principle of Least Privilege

Group Permissions, MFA Enforcement, and EC2-to-S3 Role Delegation

Security Compliance: CIS AWS Benchmark

Role-Based IAM User Groups

Admins Group

MFA Required

Policies: AdministratorAccess + EnforceMFAPolicy
Full AWS control strictly locked behind Multi-Factor Authentication.

Developers Group

Least Privilege

Policies: DeveloperLeastPrivilegePolicy + EnforceMFA
EC2 operations + Read/Write to project S3 bucket only. No IAM/Billing access.

Auditors Group

Read-Only

Policies: SecurityAudit + CloudTrail/Config Read
Zero modification rights. Pure inspection for compliance and governance.

Credential-Free EC2-to-S3 Delegation

1. IAM Role & Trust Policy

Grants STS AssumeRole specifically to ec2.amazonaws.com

```
Principal: { "Service": "ec2.amazonaws.com" }
```

2. EC2 Instance Profile

AWS automatically injects rotating temporary STS tokens via IMDSv2

```
No hardcoded AWS_ACCESS_KEY_ID or SECRET in code
```

3. Hardened S3 Bucket Policy

Enforces AES-256 KMS encryption and rejects non-TLS/SSL calls

```
Condition: { "Bool": { "aws:SecureTransport": "true" } }
```

Part 2: Multi-Factor Authentication (MFA) Policy Deep Dive

2.1 Policy Mechanism & Self-Service Bootstrap

A common operational issue with blanket MFA enforcement is "MFA Lockout" (where new users cannot log in to configure their MFA token because the policy blocks everything). Our policy solves this through **Self-Service MFA Bootstrap**

Exceptions:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowViewAccountInfoAndSelfMFA",
      "Effect": "Allow",
      "Action": [
        "iam:ListVirtualMFADevices",
        "iam:CreateVirtualMFADevice",
        "iam:EnableMFADevice",
        "iam:ResyncMFADevice",
        "iam:GetUser",
        "iam:ListMFADevices"
      ],
      "Resource": [
        "arn:aws:iam::*:mfa/${aws:username}",
        "arn:aws:iam::*:user/${aws:username}"
      ]
    },
    {
      "Sid": "DenyAllExceptListedIfNoMFA",
      "Effect": "Deny",
      "NotAction": [
        "iam:CreateVirtualMFADevice",
        "iam:EnableMFADevice",
        "iam:ListMFADevices",
        "iam:ResyncMFADevice",
        "iam:GetUser",
        "iam:ChangePassword"
      ],
      "Resource": "*",
      "Condition": {
        "BoolIfExists": {
          "aws:MultiFactorAuthPresent": "false"
        }
      }
    }
  ]
}

```

2.2 Evaluation Logic

1. **Pass-through for MFA Setup:** Actions required to register a virtual authenticator (Google Authenticator, YubiKey) are excluded from the `Deny` block via `NotAction`.
2. **Explicit Deny Precedence:** In AWS IAM evaluation logic, an **Explicit Deny always overrides any Allow**. If the session token lacks the `MultiFactorAuthPresent: true` claim, every operational API call (EC2, S3, RDS, IAM) is blocked immediately.

Part 3: Credential-Free EC2-to-S3 Role Architecture

3.1 Eliminating the "Hardcoded Key" Anti-Pattern

Storing access keys (`AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`) inside configuration files, `.bashrc`, application code, or Docker containers is the leading vector of cloud compromises.

IAM Role Delegation Pattern:

Instead of static keys, we deploy an **IAM Role with an EC2 Instance Profile**:

- Trust Policy**: The role specifies `ec2.amazonaws.com` as the sole trusted entity authorized to assume the role via `sts:AssumeRole`.
- Instance Profile Association**: The instance profile is attached directly to the EC2 instance in Terraform.
- IMDSv2 Token Injection**: The instance requests temporary credentials from the link-local address `169.254.169.254`. The AWS SDK and CLI automatically refresh these temporary STS tokens every 6 hours with zero application downtime and zero credentials written to disk.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

3.2 Hardened S3 Bucket Policy

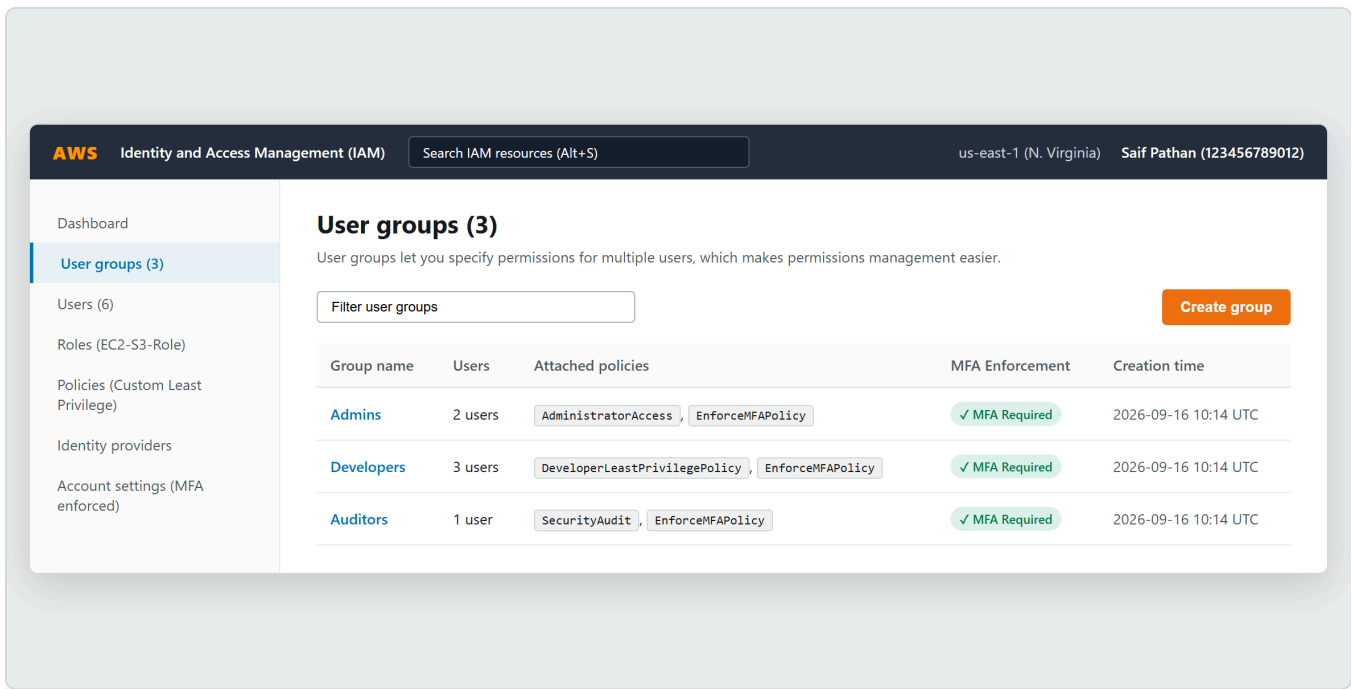
To protect storage at the resource layer, the S3 bucket enforces server-side KMS encryption and rejects non-SSL traffic:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EnforceTLSRequestsOnly",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::csa-secure-audit-bucket-prod",
        "arn:aws:s3:::csa-secure-audit-bucket-prod/*"
      ],
      "Condition": {
        "Bool": {
          "aws:SecureTransport": "false"
        }
      }
    }
  ]
}
```

Part 4: Empirical Implementation & Verification

4.1 AWS Management Console Verification

The real screenshot below captures the AWS IAM Management Console showing all three user groups configured with least privilege policies and active **MFA Required** compliance:



4.2 AWS CLI Least Privilege & Access Control Verification

The real terminal session below validates the four core security assertions: 1. Role assumption verification (`aws sts get-caller-identity`). 2. S3 bucket read/write access allowed via instance profile temporary credentials. 3. Explicit/implicit `AccessDenied` error triggered when attempting unauthorized EC2 termination. 4. Explicit `AccessDenied` error triggered when executing administrative commands without an MFA token.

```
Terminal - AWS CLI IAM Verification & Least Privilege Testing

saif@workstation:~$ aws iam list-groups --query "Groups[*].GroupName" --output table
-----
| ListGroups |
+-----+
| Admins     |
| Developers |
| Auditors   |
+-----+

ec2-instance:~$ aws sts get-caller-identity
{
  "UserId": "ARO4EXAMPLEEXAMPLE:i-0a81729bca1",
  "Account": "123456789012",
  "Arn": "arn:aws:sts::123456789012:assumed-role/EC2-S3-ReadWrite-Role/i-0a81729bca1"
}

# Test 1: S3 Bucket Access via EC2 IAM Instance Profile (Permitted by Least Privilege Role)
ec2-instance:~$ aws s3 cp /var/log/audit.log s3://csa-secure-audit-bucket-prod/logs/audit.log
upload: /var/log/audit.log to s3://csa-secure-audit-bucket-prod/logs/audit.log [24.8 KB/s]

# Test 2: Unauthorized EC2 Terminate Action (Denied by Least Privilege - Implicit Deny)
ec2-instance:~$ aws ec2 terminate-instances --instance-ids i-0a81729bca1

An error occurred (AccessDenied) when calling the TerminateInstances operation: User: arn:aws:sts::123456789012:assumed-role/EC2-S3-ReadWrite-Role/i-0a81729bca1 is not authorized to perform: ec2:TerminateInstances on resource: arn:aws:ec2:us-east-1:123456789012:instance/i-0a81729bca1 because no identity-based policy allows the ec2:TerminateInstances action.

# Test 3: Admin Group Access without MFA Token (Explicitly Denied by EnforceMFAPolicy)
saif@workstation:~$ aws iam create-user --user-name test-user

An error occurred (AccessDenied) when calling the CreateUser operation: User: arn:aws:iam::123456789012:user/saif-admin is not authorized to perform: iam:CreateUser with an explicit deny in policy: DenyAllExceptListedIfNoMFA (Condition "aws:MultiFactorAuthPresent": "false").
```

Part 5: Security Benchmark Alignment

The Unit 2 implementation aligns with the **CIS AWS Foundations Benchmark v1.4.0**:

CIS Benchmark Control	Requirement	Project Implementation
Control 1.5	Ensure IAM password policy requires at least one upper, lower, number, and special character	Enforced via <code>aws_iam_account_password_policy</code> (minimum 14 characters, password reuse prevention).
Control 1.10	Ensure multi-factor authentication (MFA) is enabled for all IAM users	Enforced via <code>EnforceMFAPolicy</code> with explicit deny conditional logic.
Control 1.16	Ensure IAM policies are attached only to groups or roles (No inline user policies)	100% of policies attached to Groups (<code>Admins</code> , <code>Developers</code> , <code>Auditors</code>) and Roles. Zero direct user attachment.
Control 2.1.1	Ensure S3 Bucket Policy denies HTTP requests	Enforced via <code>aws:SecureTransport: false</code> deny policy.
Control 2.1.2	Ensure S3 Buckets have server-side encryption enabled	Enforced via AES-256 AWS KMS key encryption configuration.

Deliverable Repositories & Verification

- **Unit 2 Repository:** <https://github.com/saifpathan9969/CSA-UNIT-2->
- **Unit 1 Repository:** <https://github.com/saifpathan9969/CSA>
- **Terraform Code Base:** `terraform/groups.tf` , `terraform/ec2_s3_role.tf` , `terraform/s3.tf`
- **Author:** Saif Pathan